

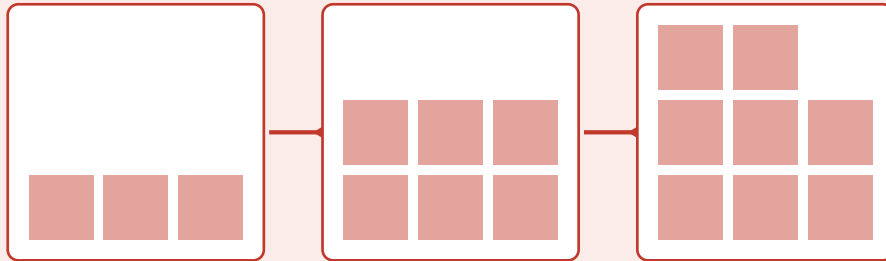
이번 리팩토링의 본질 — "빨라지는 공사"가 아니라 "맞바꾸는 공사"

1 / 5

뭘 재야 하는지는, 이 공사가 뭘 얻고 뭘 내주는지에서 저절로 정해진다

Before — 지금 구조

"한 번 방에 들인 짐은 절대 안 버린다"



게임 시작

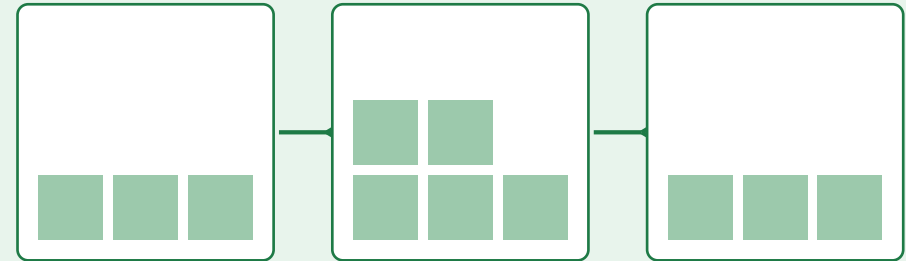
팝업 몇 번 열면

던전 다녀오면

짐(메모리)은 쌓이기만 함 — 실측: 982 → 1270 → 1287 MB

After — 목표 구조

"쓸 때만 꺼내 쓰고, 다 쓰면 반납한다"



게임 시작

팝업 여는 동안

팝업 닫으면

쓴 만큼 늘고, 끝나면 다시 줄어듦 (원상 복귀)

공짜가 아니다 — 반납하면, 다음에 쓸 때 "다시 꺼내는 시간"을 내야 한다

얻는 것 = 메모리 회수

예: 팝업 세션 영구 증가 +287MB → 0

내주는 것 = 시간 (다시 꺼내는 비용)

예: 팝업 첫 오픈이 +X ms 느려질 수 있음

→ 그래서 "좋아진 숫자(메모리)"만 재면 반쪽짜리 비교. 얻은 것과 내준 것을 둘 다 재야 비교가 완성된다.

무엇을 썰 것인가 — 지표를 역할별로 4묶음으로 나눈다

2 / 5

각 묶음은 서로 다른 질문에 답한다. 하나라도 빠지면 그 질문에 답을 못 한다

1

주 지표 — 메모리

질문: "그래서 얼마나 회수했나?"

- 팝업 세션 후 영구 증가: +287MB → 0 이 목표
- 진입 직후 Global 상주량: 982MB → 감소
- 던전에서 생긴 짐: 퇴장할 때 반납되는가
- 아틀라스 스프라이트 캐시(~300개 영구): 주인 따라 반납
- global 핸들 수: 50→58→61 처럼 오르기만 하는지, 내려오는지

개선을 증명

2

비용 지표 — 시간 · 히치

질문: "그 대가로 뭘 냈나?" (3쪽에서 자세히)

- 팝업 첫 오픈 시간: 요청 → 화면 표시까지 ms
- 오픈 순간 최악 프레임: 순간 버벅임(히치)이 생겼는가
- 씬 전환 중 메모리 최고점: 새 짐·헌 짐이 잠깐 겹치는 구간
- 같은 짐 다시 꺼낸 횟수: 반납했으니 재로드가 생김 (I/O 비용)

정직함을 증명

3

불변 지표 — 회귀 없음

질문: "고치다가 댄 걸 망가뜨리진 않았나?"

- 씬 버킷 동작: 왕복 시 완전 교체·잔존 0 (지금도 정상 → 유지 확인)
- 평시 프레임: 스코프 구조는 평상시 성능과 무관해야 정상
- Phase 2 직후: 수치가 Baseline과 완전히 같아야 성공
- (내부만 알아끼웠으니 "안 변한 것" 자체가 증거)

신뢰를 증명

4

참고 지표 — 주장에 안 씬

질문: "왜 이걸 개선 대상이 아니라고 봤나?"

- 프리로드 시간(99~177ms): 씬당 8~9개뿐 → 개선해도 티 안 남
- 앱 시작 시간: 프리로드 다이어트로 덤으로 좋아질 수 있음 (덤)
- 드로우콜: 스탠딩은 화면에 1장씩이라 아틀라스 해체 영향 없음
- → "필요 없어서 안 했다"는 것도 측정 근거로 기록해 둬

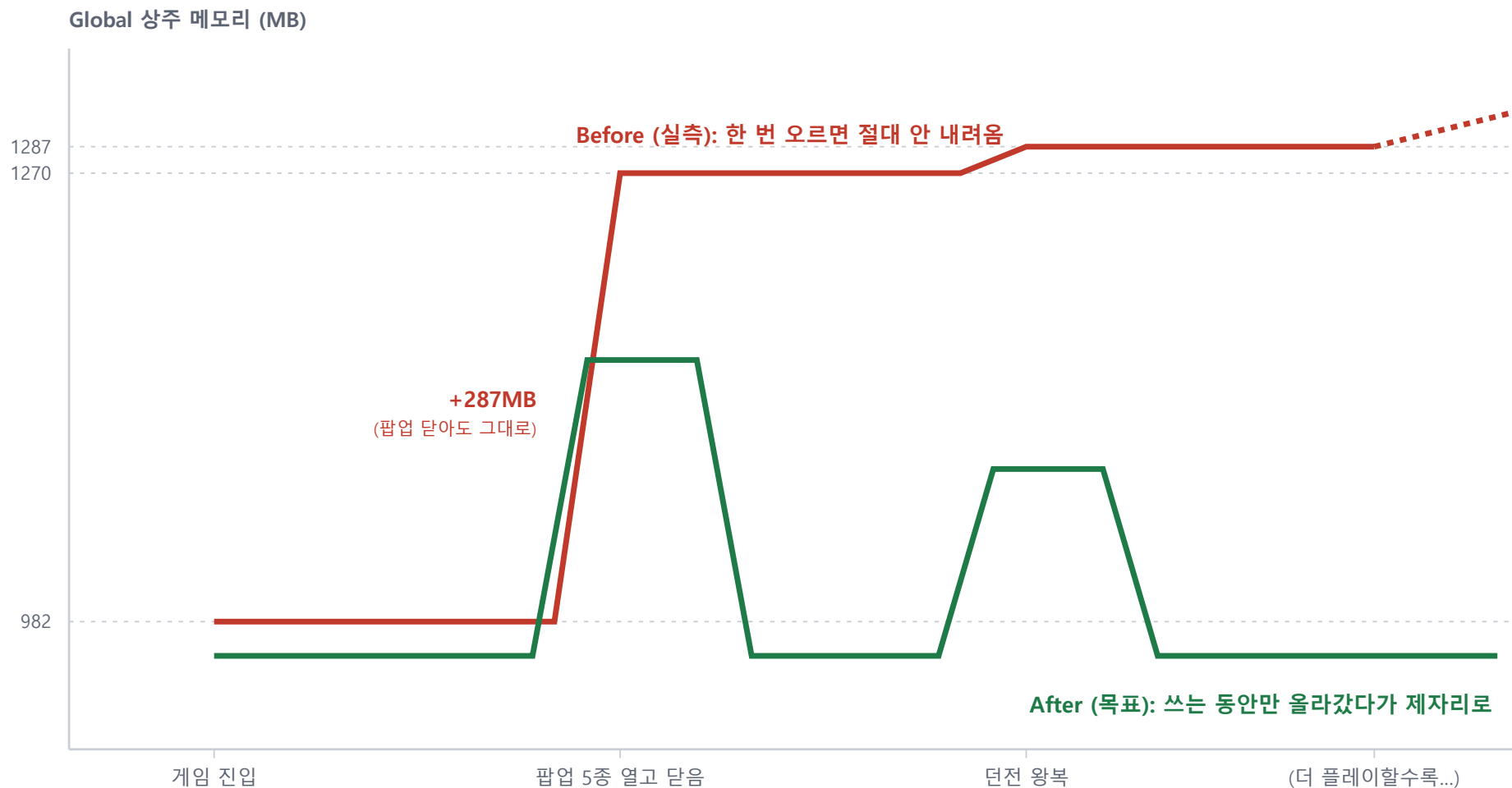
선택을 방어

한 줄 요약: 주장은 ①메모리로, 정직함은 ②히치로, 신뢰는 ③불변 지표로. ④는 "안 한 이유"의 근거.

① 주 지표(메모리) — 스냅샷 한 장이 아니라 "흐름 그래프"로 비교

3 / 5

핵심 증거는 "오르기만 하던 선이, 이제는 내려오기도 한다"는 모양의 변화



- 비교 방법: Before와 완전히 같은 순서로 행동(진입 → 팝업 5종 → 던전 왕복)하면서 같은 도구(디버그 창)로 기록 → 두 그래프를 겹쳐 봄
- 왜 "비율(global vs scene)"이 아닌가: 정당하게 영구인 리소스는 고치고 나서도 global에 남는다. 비율은 여전히 global 쪽으로 기울 수 있음
- 진짜 목표는 두 가지 — (1) 선이 내려오기도 하는가(단조 증가 소멸) (2) 시작점 자체가 낮아졌는가(프리로드 다이어트, 982 → ↓)
- 주의: 에디터 측정치는 실제보다 약 2배 부풀려짐 → 절대값 말고 "모양·증감"으로 비교, 최종 수치는 개발 빌드에서 1회 실측

② 비용 지표 — 비동기는 일을 없애지 않고, 떨어지는 "타이밍"만 옮긴다

4 / 5

반납한 짐은 다시 꺼내야 한다. 그 비용이 어느 프레임에 떨어지는지를 재는 것이 비용 측정

도식 A. 같은 양의 일, 다른 떨어지는 자리 — "히치(순간 버벅임)"의 정체

로드+생성+UI조립을 한 프레임에

전부 한 번에 (동기)



나눠서 (비동기)

파일 읽기는 뒤(백그라운드)에서
생성+UI조립은 여전히 메인 스레드!



→ 비동기여도 히치가 0이 되진 않는다. 그래서 "최악 프레임 시간"을 실제로 재야, 비동기가 히치를 흡수했는지 알 수 있다.

도식 B. 씬 전환 순서 교체의 비용 — 메모리 "최고점"이 잠깐 올라간다

Before: 현 짐부터 버리고 → 새 짐 들임



최고점 낮음 (겹치는 구간 없음)

대신 화면에 아직 보이는 물건의 재료를 먼저 버리는
위험한 순간이 존재 (핑크 텍스처 사고 가능성)

→ 안전 대신 최고점을 낮춘 상태

After: 새 짐 들인 뒤 → 현 짐 버림 (안전)



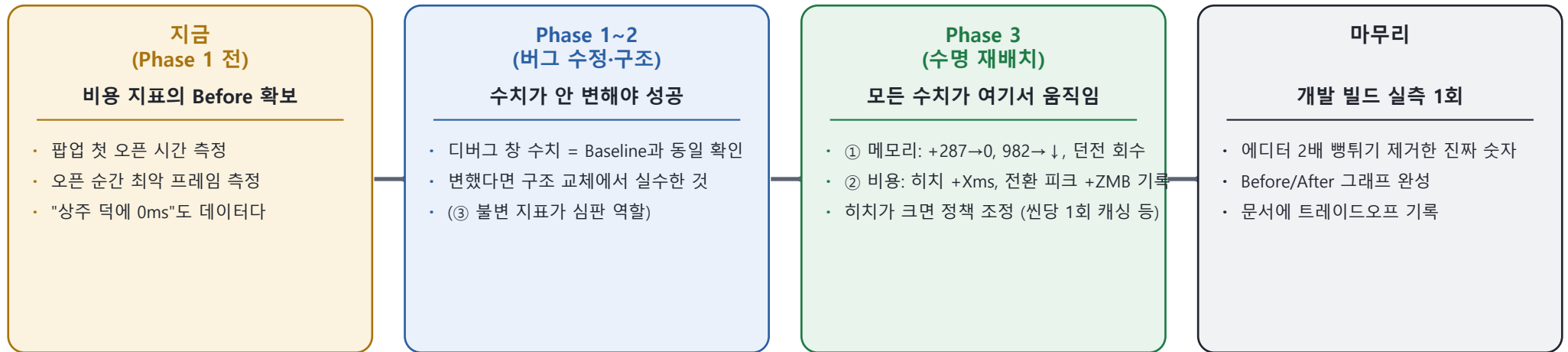
이 겹침 구간만큼 메모리 최고점이 +XMB 올라감
= 안전(사고 창 제거)을 사기 위해 지불하는 비용
→ 미리 재서 "알고 지불했다"고 기록해 두는 게 핵심

→ 최고점 대신 안전을 산 상태 (측정으로 결정)

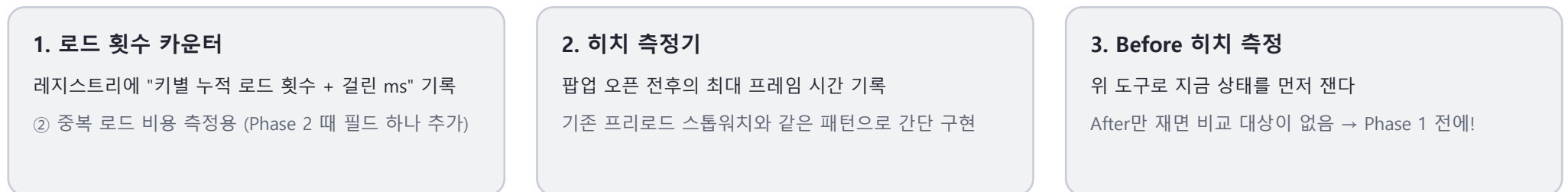
정리 — 언제, 무엇으로, 무엇을 재는가

5 / 5

측정 시나리오는 Baseline과 완전히 동일하게. 도구가 같아야 비교가 성립한다



측정 준비물 — 지금 도구에 3가지만 추가하면 됨



완성된 비교의 모습 (이 한 문단을 채우는 것이 목표)

"메모리 X MB를 회수했고, 그 대가인 히치는 썬당 첫 1회 +Y ms로 제한했으며, 전환 피크 +Z MB는 안전의 비용으로 수용했다.
평시 성능과 썬 단위 해제는 변하지 않았음을 같은 도구로 증명했다."